DATAPROTECT
Security is our commitment

RAPPORT DE STAGE PFA

BU Offensive Security – DATAPROTECT

Cloud Risk Assessment Tool

Sujet

Développement d'un outil de découverte et de scoring de risque sur un environnement cloud fictif (AWS/Azure), avec détection des mauvaises configurations IAM, buckets exposés et assets orphelins.

Réalisé par :

KHALED Chahid

Encadré par :

M. ABDESSADKI Imad
Mme. AKROUT Salma

2026

Remerciements

Je tiens à exprimer mes sincères remerciements à **DATAPROTECT** pour m'avoir accueilli dans le cadre de ce stage PFA et pour m'avoir offert un environnement professionnel favorable à l'apprentissage et à la mise en pratique de compétences en cybersécurité.

J'adresse une reconnaissance particulière à mes encadrants, **M. ABDESSADKI Imad** et **Mme AKROUT Salma**, pour leur accompagnement, leur disponibilité, leurs orientations et les retours apportés tout au long du stage et de la réalisation de ce projet.

Je remercie également l'**École Nationale des Sciences Appliquées d'Oujda (ENSAO)**, ainsi que l'ensemble de ses enseignants, pour la formation académique et technique qui a constitué la base nécessaire à la conduite de ce travail.

Enfin, je remercie toutes les personnes ayant contribué, directement ou indirectement, au bon déroulement de ce stage et à l'aboutissement de ce rapport.

Table des acronymes

Acronyme	Signification
API	Application Programming Interface
ARM	Azure Resource Manager
AWS	Amazon Web Services
BU	Business Unit
CLI	Command-Line Interface
CSPM	Cloud Security Posture Management
EBS	Elastic Block Store
EC2	Elastic Compute Cloud
ENI	Elastic Network Interface
HTML	HyperText Markup Language
HTTP/HTTPS	HyperText Transfer Protocol / HTTP Secure
IAM	Identity and Access Management
JSON	JavaScript Object Notation
MFA	Multi-Factor Authentication
NIC	Network Interface Card
NSG	Network Security Group
PFA	Projet de Fin d'Année
RBAC	Role-Based Access Control
REST	Representational State Transfer
S3	Simple Storage Service
SDK	Software Development Kit
STS	Security Token Service
TLS	Transport Layer Security
VM	Virtual Machine
VNet	Virtual Network
VPC	Virtual Private Cloud

Résumé

Ce rapport présente la conception et la réalisation de **Cloud Risk Assessment Tool**, développé dans le cadre d'un stage PFA au sein de la **BU Offensive Security de DATAPROTECT**. Le projet vise à automatiser un cycle de *Cloud Security Assessment* sur deux environnements fictifs représentant AWS et Azure. L'outil suit une architecture commune fondée sur quatre étapes : **Discovery**, **Security Analysis**, **Risk Scoring** et **Reporting**. La version AWS s'appuie sur **LocalStack** et `boto3`, tandis que la version Azure utilise **Topaz**, une API REST locale et `topaz-sdk`. Les contrôles implémentés portent principalement sur les mauvaises configurations IAM/RBAC, l'exposition publique des espaces de stockage et la détection d'assets orphelins, avec quelques contrôles réseau complémentaires côté Azure. Des laboratoires reproductibles contenant des ressources sûres et vulnérables permettent de valider les détections, de contrôler les faux positifs et de vérifier le calcul des scores. Les résultats sont enfin exportés en JSON et dans un rapport HTML exploitable, ce qui fournit une base technique extensible pour l'ajout de nouvelles ressources, règles de sécurité ou méthodes de scoring.

Mots-clés : Cloud Security Assessment, AWS, Azure, IAM, RBAC, LocalStack, Topaz, scoring de risque, sécurité Cloud.

Abstract

This report presents the design and implementation of the **Cloud Risk Assessment Tool**, developed during a PFA internship within **DATAPROTECT's Offensive Security Business Unit**. The project aims to automate a *Cloud Security Assessment* cycle across two fictitious environments representing AWS and Azure. The tool follows a common four-stage architecture : **Discovery**, **Security Analysis**, **Risk Scoring**, and **Reporting**. The AWS implementation relies on **LocalStack** and `boto3`, while the Azure implementation uses **Topaz**, a local REST API, and `topaz-sdk`. The implemented controls mainly address IAM/RBAC misconfigurations, publicly exposed storage resources, and orphaned assets, with additional network-related controls on the Azure side. Reproducible laboratories containing both secure and intentionally vulnerable resources are used to validate detections, check false positives, and verify risk-score calculations. The results are exported as structured JSON files and as an HTML report, providing an extensible technical foundation for adding new resources, security rules, and scoring models.

Keywords : Cloud Security Assessment, AWS, Azure, IAM, RBAC, LocalStack, Topaz, risk scoring, cloud security.

Table des matières

Remerciements **i**

Table des acronymes **ii**

Résumé **iii**

Abstract **iv**

Liste des figures **viii**

Liste des tableaux **ix**

1 Contexte général du projet **1**
1.1 Introduction générale 1
1.2 Présentation de l'organisme d'accueil 1
1.2.1 DATAPROTECT 1
1.2.2 Business Unit Offensive Security 2
1.3 Présentation du sujet 2
1.4 Définitions et concepts clés 3
1.5 Méthodologie adoptée 3
1.6 Objectifs 3
1.7 Périmètre 4

2 Étude technique et environnement de test **6**
2.1 Architecture générale 6
2.2 Workflow 7
2.3 Environnement fictif AWS : LocalStack 7
2.3.1 Composition du laboratoire AWS 8
2.4 Environnement fictif Azure : Topaz 9
2.4.1 Accès local à Topaz 10
2.4.2 Composition du laboratoire Azure 11
2.5 Discovery 12
2.6 Security Analysis 12
2.7 Risk Scoring 13

2.8 Reporting 13

3 Outils et technologies utilisés 15
3.1 Python 15
3.2 Docker et Docker Compose 15
3.3 AWS, LocalStack et boto3 15
3.4 Azure, Topaz, topaz-sdk et Requests 16
3.5 PowerShell et automatisation locale 16
3.6 Formats JSON et HTML 16

4 Développement de l'implémentation AWS 17
4.1 Architecture du projet 17
4.2 Discovery 18
4.2.1 Découverte IAM 19
4.2.2 Découverte S3 19
4.2.3 Découverte EC2 20
4.3 Détection des mauvaises configurations IAM 20
4.4 Détection des buckets S3 exposés 21
4.5 Détection des assets orphelins 22
4.6 Risk Scoring 23
4.7 Exécution 24
4.8 Reporting 25

5 Développement de l'implémentation Azure 27
5.1 Architecture du projet 27
5.2 Discovery 28
5.2.1 Connexion aux API Topaz 28
5.2.2 Organisation de la découverte 29
5.2.3 Identity et RBAC 29
5.2.4 Storage 29
5.2.5 Compute et Network 30
5.3 Détection des mauvaises configurations RBAC 30
5.4 Détection du stockage exposé 31
5.4.1 Contrôles réseau complémentaires 32
5.5 Détection des assets orphelins 32
5.6 Risk Scoring 33
5.7 Exécution 34
5.8 Reporting 35

6 Tests, résultats et constats 37
6.1 Scénarios de test 37
6.1.1 Scénario AWS 37
6.1.2 Scénario Azure 38

6.2 Résultats AWS 39
6.3 Résultats Azure 40
6.4 Validation des détections 41
6.4.1 Pour AWS, la matrice principale est : 41
6.4.2 Pour Azure : 42
6.5 Constats principaux 43

7 Conclusion générale **44**

Perspectives **46**

Annexes **47**

Webographie / Bibliographie **48**

Table des figures

Figure 2.1 8
Figure 2.2 9
Figure 2.3 10
Figure 2.4 12

Figure 4.1 26
Figure 4.2 26

Figure 5.1 35
Figure 5.2 36

Liste des tableaux

1.1 Périmètre fonctionnel comparé des implémentations AWS et Azure . . 4

2.1 Fichiers principaux du laboratoire AWS LocalStack 7
2.2 Fichiers principaux du laboratoire Azure Topaz 10
2.3 Affectations RBAC du laboratoire Azure 11
2.4 Structure d'un finding de sécurité 12

4.1 Principaux composants de l'implémentation AWS 17
4.2 Règles de détection des mauvaises configurations IAM 20
4.3 Règles de détection des buckets S3 exposés 22
4.4 Règles de détection des assets orphelins AWS 23
4.5 Bonus contextuels appliqués au scoring AWS 23
4.6 Seuils de sévérité du scoring AWS 24
4.7 Scripts PowerShell de l'implémentation AWS 24

5.1 Principaux composants de l'implémentation Azure 27
5.2 Règles de détection des mauvaises configurations RBAC 30
5.3 Règles de détection du stockage Azure exposé 31
5.4 Règles de détection des assets orphelins Azure 32
5.5 Scores de base du modèle de scoring Azure 33
5.6 Seuils de sévérité du scoring Azure 33
5.7 Scripts PowerShell de l'implémentation Azure 34

6.1 Scénarios fonctionnels de test AWS 38
6.2 Scénarios fonctionnels de test Azure 38
6.3 Résultats de référence de l'implémentation AWS 39
6.4 Synthèse des résultats de référence AWS 39
6.5 Résultats de référence de l'implémentation Azure 40
6.6 Synthèse des résultats de référence Azure 40
6.7 Matrice de validation des détections AWS 41
6.8 Matrice de validation des détections Azure 42

Chapitre 1

Contexte général du projet

1.1 Introduction générale

La transformation numérique des organisations s'accompagne d'un recours croissant aux services Cloud pour héberger des applications, stocker des données et fournir des ressources informatiques à la demande. Cette souplesse introduit toutefois un enjeu majeur : la sécurité dépend non seulement des mécanismes fournis par le fournisseur Cloud, mais aussi de la manière dont les identités, les autorisations, le stockage et les ressources sont configurés.

Dans ce contexte, l'évaluation régulière de la posture de sécurité permet d'identifier des configurations susceptibles d'augmenter la surface d'exposition ou d'accorder des privilèges excessifs. Le présent projet s'inscrit dans cette logique en proposant un outil capable de collecter automatiquement des informations sur un environnement Cloud, d'appliquer des règles de sécurité explicites, de prioriser les constats par un score de risque et de restituer les résultats sous une forme exploitable.

Le travail a été réalisé dans le cadre d'un stage PFA au sein de la Business Unit Offensive Security de DATAPROTECT. Afin de disposer d'un cadre de développement contrôlé et reproductible, les expérimentations s'appuient sur des environnements locaux fictifs représentant AWS et Azure, sans dépendre de comptes Cloud de production.

1.2 Présentation de l'organisme d'accueil

1.2.1 DATAPROTECT

Créée en 2009 et basée à Casablanca, **DATAPROTECT** est une entreprise spécialisée en cybersécurité. Son activité couvre plusieurs pôles complémentaires, notamment la sécurité offensive, le CyberSOC et les services managés, la gouvernance, le risque et la conformité, la cyberdéfense, la résilience ainsi que les services liés à la sécurité Cloud. Cette approche lui permet d'intervenir aussi bien sur des enjeux techniques que sur des

problématiques de gouvernance et de maîtrise du risque [1, 3].

1.2.2 Business Unit Offensive Security

La **BU Offensive Security** est consacrée aux activités d'évaluation technique de la sécurité. Les prestations présentées par DATAPROTECT reposent notamment sur des audits et des simulations d'attaques éthiques visant à identifier les vulnérabilités d'un système d'information et à proposer des actions correctives adaptées [2].

Le stage s'inscrit directement dans ce contexte : le projet développé ne réalise pas un test d'intrusion classique sur une application ou un réseau, mais automatise une partie de l'analyse de posture de sécurité d'environnements Cloud. La logique retenue reste cohérente avec une démarche d'audit technique : collecte des faits, identification de conditions à risque, qualification de la criticité et production d'un rapport exploitable.

1.3 Présentation du sujet

Le projet **Cloud Risk Assessment Tool** a pour objectif de développer un outil capable de découvrir des ressources Cloud, d'analyser leur configuration de sécurité et de produire un score de risque exploitable. La solution cible deux fournisseurs Cloud, **Amazon Web Services (AWS)** et **Microsoft Azure**, mais elle est volontairement testée dans des environnements locaux fictifs afin d'éviter l'utilisation de comptes Cloud réels pendant le développement.

Le sujet est centré sur trois familles de risques :

— les **mauvaises configurations IAM / RBAC**, qui peuvent attribuer des privilèges excessifs ou exposer des mécanismes d'authentification insuffisamment protégés ;
— les **buckets ou espaces de stockage exposés**, lorsqu'une configuration autorise un accès anonyme ou public non souhaité ;
— les **assets orphelins**, c'est-à-dire des ressources présentes dans l'environnement mais non rattachées à une charge de travail active.

L'outil est organisé comme une chaîne de traitement indépendante de l'interface Cloud : une première phase collecte les ressources, une deuxième applique des règles de sécurité, une troisième calcule le niveau de risque, puis une dernière phase génère les fichiers de sortie et le rapport HTML.

L'implémentation AWS utilise **LocalStack** afin d'émuler localement les services nécessaires au projet. L'implémentation Azure utilise **Topaz**, qui fournit une API locale reproduisant les comportements nécessaires à la découverte des ressources Azure utilisées par l'outil. Dans les deux cas, les environnements sont exécutés avec Docker et sont initialisés avec des ressources volontairement sûres ou vulnérables afin de valider les mécanismes de détection.

1.4 Définitions et concepts clés

Plusieurs notions structurent le projet :

- **Cloud Security Assessment** : démarche consistant à observer la configuration d'un environnement Cloud afin d'identifier des conditions susceptibles d'affecter sa sécurité ;
- **IAM / RBAC** : mécanismes de gestion des identités, des rôles et des autorisations. IAM est utilisé côté AWS, tandis qu'Azure RBAC gère l'attribution de rôles à des principaux sur un scope donné ;
- **stockage exposé** : espace de stockage dont la configuration permet un accès public ou anonyme non souhaité, par exemple un bucket S3 ou un Blob Container ;
- **asset orphelin** : ressource toujours présente dans l'environnement mais qui n'est plus rattachée à une charge de travail active, comme un volume, une interface réseau ou une adresse IP non utilisée ;
- **finding** : constat structuré produit par une règle de sécurité et contenant notamment l'identifiant de la règle, la ressource concernée, les preuves, la recommandation et les paramètres de scoring ;
- **Risk Scoring** : mécanisme de priorisation attribuant un score et un niveau de sévérité aux findings afin de faciliter leur traitement.

1.5 Méthodologie adoptée

La méthodologie suivie repose sur une démarche incrémentale et reproductible. Elle commence par la construction d'un laboratoire local contenant à la fois des ressources normales et des configurations volontairement risquées. Les données sont ensuite collectées sans interprétation de sécurité, puis normalisées dans un inventaire commun. Des règles déterministes sont appliquées à cet inventaire pour produire des findings, avant le calcul d'un score de risque et la génération des rapports.

La validation s'effectue en comparant les constats produits avec les cas attendus du laboratoire. Les ressources sûres jouent le rôle de contrôles négatifs afin de vérifier que la présence d'une ressource ne suffit pas à générer un finding. Cette séparation entre Discovery, Analysis, Scoring et Reporting constitue le principe méthodologique central du projet.

1.6 Objectifs

L'objectif principal n'est pas de construire un scanner Cloud universel, mais une base technique fonctionnelle et extensible qui démontre un cycle complet de **Cloud Security Assessment**. La solution doit être capable de passer d'un environnement brut à un résultat directement interprétable sans intervention manuelle entre les différentes étapes.

Les objectifs fonctionnels sont les suivants :

1. **Découvrir automatiquement les ressources utiles à l'évaluation.** Pour AWS, cela concerne principalement IAM, S3 et EC2. Pour Azure, la découverte couvre les identités, RBAC, Storage, Compute, Network et les groupes de ressources.
2. **Normaliser les informations collectées.** Les API Cloud retournent des structures différentes selon les services. La phase Discovery doit donc produire un inventaire cohérent, exploitable par les modules d'analyse.
3. **Appliquer des règles de détection explicites.** Chaque problème de sécurité doit correspondre à une règle identifiable, avec un identifiant, une description, des preuves et une recommandation.
4. **Attribuer un score de risque.** Le score doit tenir compte de la gravité technique du finding et, lorsque l'information est disponible, de facteurs de contexte comme l'exposition Internet, la criticité ou la sensibilité des données.
5. **Générer un reporting exploitable.** Les résultats doivent être disponibles sous une forme lisible par une personne, mais aussi sous une forme structurée pouvant être réutilisée par un autre programme.
6. **Permettre une exécution simple.** Des scripts PowerShell doivent pouvoir prendre en charge l'installation locale, le démarrage du laboratoire, le scan, la réinitialisation et l'arrêt de l'environnement.
7. **Conserver une séparation claire des responsabilités.** Discovery ne doit pas décider si une ressource est vulnérable ; Analysis ne doit pas recréer la logique d'accès aux API ; Scoring doit rester indépendant des mécanismes de découverte ; Reporting ne doit pas modifier les findings.

Cette séparation facilite la maintenance du projet. Une nouvelle règle de sécurité peut être ajoutée dans Analysis sans réécrire le mécanisme de connexion au Cloud, et un nouveau format de rapport peut être ajouté sans modifier les règles de détection.

1.7 Périmètre

Le périmètre du projet est volontairement limité aux éléments nécessaires à la démonstration d'un outil de découverte et de scoring multi-Cloud. Il couvre deux implémentations séparées mais construites autour de la même logique fonctionnelle :

Table 1.1 – Périmètre fonctionnel comparé des implémentations AWS et Azure

Domaine	AWS	Azure
Identités et autorisations	IAM Users, Groups, Roles, Policies	Users, Service Principals, Role Assignments
Stockage	S3 Buckets	Storage Accounts, Blob Containers

Domaine	AWS	Azure
Compute	EC2 Instances, EBS Volumes	Virtual Machines, Managed Disks
Réseau utile au diagnostic	Elastic IP, Network Interfaces	Public IP, NIC, NSG, VNet
Environnement de test	LocalStack	Topaz
Reporting	JSON + HTML	JSON + HTML

Le projet ne cherche pas à reproduire tous les services disponibles dans AWS ou Azure. Il ne couvre pas non plus l'ensemble des contrôles CIS, des exigences de conformité ou des techniques CSPM disponibles dans des produits spécialisés. Le périmètre est concentré sur les familles de risques prévues dans le sujet et sur quelques contrôles complémentaires lorsque les données nécessaires sont déjà collectées.

L'utilisation de LocalStack et de Topaz signifie également que l'évaluation est réalisée sur un **laboratoire local contrôlé**. Le code est structuré de manière à rester proche des API Cloud, mais le laboratoire n'a pas vocation à reproduire parfaitement toutes les particularités opérationnelles d'un tenant ou d'un compte de production.

Chapitre 2

Étude technique et environnement de test

2.1 Architecture générale

La solution suit une architecture en pipeline. Chaque étape reçoit le résultat de l'étape précédente et ajoute une couche d'information sans modifier la responsabilité des autres modules.

```
Cloud fictif -> Discovery -> Inventaire structuré
-> Security Analysis -> Findings -> Risk Scoring
-> Findings scorés -> Reporting -> JSON / HTML
```

Cette architecture permet de représenter le fonctionnement général avec quatre blocs principaux :

- **Discovery** : interroge les API et construit l'inventaire ;
- **Security Analysis** : transforme des configurations observées en findings de sécurité ;
- **Risk Scoring** : attribue une valeur et une sévérité aux findings ;
- **Reporting** : sauvegarde les données et produit une représentation destinée à l'utilisateur.

Le pipeline est orchestré par `pipeline.py` dans les deux projets. Dans AWS, la classe `ScanPipeline` instancie `AWSClientFactory`, `DiscoveryManager`, `AnalysisManager`, `RiskScoringEngine` et `ReportGenerator`. Dans Azure, la classe `Pipeline` enchaîne de la même manière `DiscoveryManager`, `AnalysisManager`, `ScoringEngine` et `ReportGenerator`.

Cette similarité est importante : les technologies de connexion et les modèles de ressources diffèrent, mais le cycle d'évaluation reste identique. Il devient donc possible d'ajouter un autre provider sans changer la logique conceptuelle du projet.

2.2 Workflow

Le workflow d'exécution ne commence pas directement dans le code de Discovery. Avant le scan, l'environnement de test doit être disponible et contenir des ressources. Les scripts PowerShell jouent ici un rôle d'orchestration : ils préparent Python, démarrent Docker, construisent le laboratoire puis appellent la commande de scan.

```
PowerShell / quickstart.ps1 -> Environnement Python -> Docker Compose
-> LocalStack / Topaz -> Initialisation du laboratoire -> Scan Python
-> Discovery -> Analysis -> Scoring -> Reporting -> output/
```

Le workflow présente deux avantages. Premièrement, l'utilisateur peut lancer une démonstration complète avec une commande unique. Deuxièmement, les scripts séparés permettent de ne relancer que l'étape nécessaire : démarrer le laboratoire, lancer un nouveau scan, reconstruire l'environnement ou l'arrêter.

Le code applicatif reste néanmoins utilisable indépendamment des scripts. Les scripts constituent une couche de commodité et d'automatisation ; le cœur de l'outil reste le package Python et son interface CLI.

2.3 Environnement fictif AWS : LocalStack

L'implémentation AWS est testée avec **LocalStack**, exécuté dans un conteneur Docker. Le projet ne contacte donc pas un compte AWS réel pendant la démonstration. Le code Python utilise les mêmes familles d'API que dans AWS, mais les clients `boto3` sont redirigés vers l'endpoint local configuré dans `Settings` :

```
http://localhost:4566
```

Le fichier `localstack/compose.yml` expose le port `4566` uniquement sur l'interface locale et active les services nécessaires au projet : `s3`, `iam`, `sts` et `ec2`. La configuration utilise une région principale `us-east-1`, désactive la persistance pour reconstruire facilement un laboratoire propre et utilise le mode EC2 simulé adapté à une démonstration locale.

Le répertoire `localstack/` contient trois éléments :

Table 2.1 – Fichiers principaux du laboratoire AWS LocalStack

Fichier	Rôle
`compose.yml`	Décrit le service Docker, les ports, les variables d'environnement et le healthcheck
`Dockerfile`	Construit l'IMAGE du laboratoire à partir de l'image LocalStack définie par le projet

Fichier	Rôle
`init-aws.sh`	Initialise les ressources AWS fictives et introduit les configurations sûres et vulnérables

Le laboratoire est donc reproductible. Une reconstruction recrée le même ensemble de ressources de référence, ce qui permet de comparer les résultats du scan avec un état attendu.

```
Docker Desktop
      |
LocalStack
      |
API locale : http://localhost:4566
      |
Python + boto3
      |
Services AWS : IAM / S3 / EC2
      |
Discovery
```

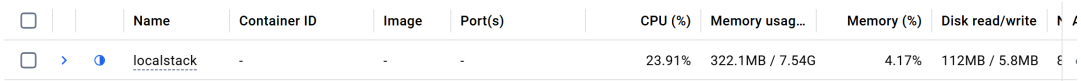


Figure 2.1

2.3.1 Composition du laboratoire AWS

Le script `init-aws.sh` ne crée pas uniquement des ressources vulnérables. Il crée également des ressources normales afin de vérifier que l'outil ne signale pas systématiquement tout ce qu'il découvre.

Pour S3, trois buckets sont préparés :

- `crl-private-data` : bucket privé avec les quatre contrôles Block Public Access activés ;
- `crl-public-documents` : bucket contenant une policy autorisant `s3:GetObject` à `Principal: "*"` et dont les protections publiques sont désactivées ;
- `crl-public-upload` : bucket permettant `s3:PutObject` à un principal public.

Les buckets portent des tags utiles au scoring, notamment `Criticality` et `DataClassification`. Cette information ne sert pas à déterminer qu'un bucket est public ; elle sert à augmenter la priorité d'un finding lorsque la ressource est critique ou contient des données sensibles.

Pour IAM, le laboratoire crée :

— `crl-safe-user`, avec une policy limitée à la lecture du bucket privé ;
— `crl-risky-admin`, avec une policy inline autorisant `Action: "*"` sur `Resource: "*"` ;
— `crl-console-no-mfa`, disposant d'un profil de connexion console mais sans MFA ;
— `crl-public-trust-role`, dont la trust policy autorise un principal wildcard à effectuer `sts:AssumeRole`.

Le laboratoire EC2 contient une petite infrastructure de référence : un VPC, un subnet, un security group, une instance utilisée, un volume attaché et une Elastic IP associée. À côté de ces ressources normales, le script crée trois ressources orphelines :

— un volume EBS `crl-orphan-volume` non attaché ;
— une Elastic IP `crl-orphan-eip` non associée ;
— une interface réseau `crl-orphan-eni` détachée.

Ce mélange de ressources sûres et risquées est important : la validation ne consiste pas seulement à vérifier que des findings apparaissent, mais que les règles distinguent correctement les états attendus.

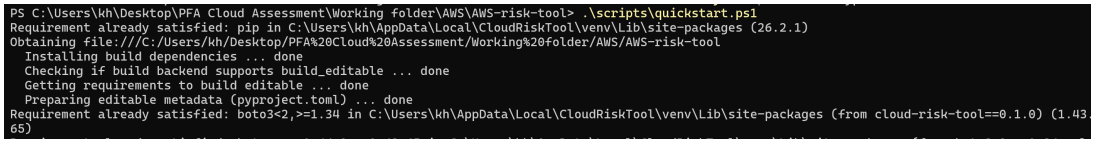


Figure 2.2

2.4 Environnement fictif Azure : Topaz

La partie Azure utilise **Topaz** comme environnement Cloud local. Le service est lancé dans Docker à partir de l'image `thecloudtheory/topaz-host` et expose les interfaces nécessaires au projet sans exiger un abonnement Azure réel.

Le fichier `topaz/compose.yml` définit un service `topaz`, un volume persistant et deux ports utilisés par l'application :

— `8899` pour les appels de type Azure Resource Manager et les opérations de contrôle ;
— `8891` pour le data plane Storage utilisé pour les Blob Containers.

Le conteneur est lancé avec une subscription fictive par défaut :

`11111111-1111-1111-1111-111111111111`

Dans la configuration Python, l'endpoint de contrôle est `https://localhost:8899`. Les requêtes envoient cependant le header `Host: topaz.local.dev:8899`, et le scope utilisé pour le token local est construit autour de `topaz.local.dev`. Le

data plane Storage utilise `https://localhost:8891` avec un nom d'hôte de type `<storage-account>.blob.storage.topaz.local.dev:8891`.

Le répertoire `topaz/` contient :

Table 2.2 – Fichiers principaux du laboratoire Azure Topaz

Fichier	Rôle
`compose.yml`	Définition du conteneur, des ports et du volume Topaz
`Dockerfile`	Construction de l'image locale à partir de `thecloudtheory/topaz-host`
`init-azure.py`	Point d'entrée Python vers `azure_risk_tool.lab.main`
`init-azure.sh`	Wrapper permettant de lancer l'initialisation avec Python

La création réelle du laboratoire se trouve dans `lab.py`. Ce choix rend la définition du lab plus riche qu'un simple script shell : les mêmes classes de connexion que celles utilisées par le projet peuvent servir à créer les ressources Azure fictives.

```
Docker Desktop -> Topaz
                  |-- ARM / Resource API  : HTTPS 8899
                  `-- Blob Storage API    : HTTPS 8891

lab.py -> APIs Topaz
Discovery -> azure.py -> APIs Topaz
```



Figure 2.3

2.4.1 Accès local à Topaz

Sur Windows, le projet prévoit une association locale du nom `topaz.local.dev` vers `127.0.0.1`. Cette configuration est effectuée une seule fois avec des droits administrateur :

```
Add-Content -Path "$env:SystemRoot\System32\drivers\etc\hosts" -Value "`n127.0.0.1 t
ipconfig /flushdns
```

La résolution et l'ouverture du port peuvent ensuite être contrôlées avec :

```
ping topaz.local.dev
Test-NetConnection topaz.local.dev -Port 8899
```

Cette étape est distincte du scan lui-même : elle prépare simplement le poste afin que les noms d'hôte utilisés par Topaz et le projet puissent être résolus localement.

2.4.2 Composition du laboratoire Azure

`LabBuilder` construit un environnement suffisamment varié pour tester les règles de sécurité. Au lancement, il attend que Topaz soit disponible, crée les ressources, écrit le manifeste dans `output/lab-manifest.json` puis affiche les cas attendus.

Deux Resource Groups sont créés :

— `rg-production` ;
— `rg-operations`.

Le laboratoire crée ensuite quatre identités : deux utilisateurs et deux service principals. `Alice Admin` et `Bob Reader` représentent des utilisateurs, tandis que `sp-deployment` et `sp-monitoring` représentent des identités applicatives.

Les Role Assignments sont choisis pour produire à la fois des cas à risque et des cas normaux :

Table 2.3 – Affectations RBAC du laboratoire Azure

Principal	Rôle	Scope	Attendu
Alice Admin	Owner	Subscription	Critique
sp-deployment	Contributor	Subscription	Risqué
Bob Reader	Reader	`rg-production`	Cas normal
sp-monitoring	Reader	`rg-production`	Cas normal

Le stockage comprend deux comptes. `stpublicdata` autorise l'accès public Blob et contient un container `public-data` configuré au niveau `container`, avec un fichier de démonstration `customers.csv`. `stprivatebackups` interdit `allowBlobPublicAccess` et contient un container privé.

La partie réseau et compute crée notamment :

— un Virtual Network et un subnet ;
— un NSG `nsg-public-admin` autorisant RDP 3389 et SSH 22 depuis Internet ;
— un NSG interne avec uniquement HTTPS depuis le réseau privé ;
— une Public IP associée à la VM et une Public IP inutilisée ;
— une NIC utilisée par la VM et une NIC non attachée ;
— un disque de données attaché et un disque `disk-old-backup` non attaché ;
— une VM `vm-web-01` utilisant la NIC et le disque actifs.

Le manifeste du laboratoire annonce explicitement les cas de sécurité attendus : Owner au scope subscription, Contributor au scope subscription, Blob container public, Storage Account autorisant le public Blob, exposition RDP/SSH, disque non attaché, NIC non attachée et Public IP non attachée.

```
PS C:\Users\kh\Desktop\PFA Cloud Assessment\Working folder\azure\azure-risk-tool> .\scripts\quickstart.ps1
Requirement already satisfied: pip in .\.venv\Lib\site-packages (26.2.1)
Obtaining file:///C:/Users/kh/Desktop/PFA%20Cloud%20Assessment/Working%20folder/azure/azure-risk-tool
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Requirement already satisfied: requests<3,>=2.31 in .\.venv\Lib\site-packages (from azure-risk-tool==0.1.1) (2.34.2)
Requirement already satisfied: topaz-sdk in .\.venv\Lib\site-packages (from azure-risk-tool==0.1.1) (0.1.2)
Requirement already satisfied: charset_normalizer<4,>=2 in .\.venv\Lib\site-packages (from requests<3,>=2.31->azure-risk
-tool==0.1.1) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in .\.venv\Lib\site-packages (from requests<3,>=2.31->azure-risk-tool==0.1.1
) (3.18)
Requirement already satisfied: urllib3<3,>=1.26 in .\.venv\Lib\site-packages (from requests<3,>=2.31->azure-risk-tool==0
```

Figure 2.4

2.5 Discovery

La phase **Discovery** constitue la frontière entre l'environnement Cloud et le moteur d'analyse. Elle ne décide pas si une configuration est bonne ou mauvaise. Son rôle est de répondre à trois questions : quelles ressources existent, quelles sont leurs propriétés pertinentes, et quelles relations existent entre elles ?

Dans AWS, les clients sont créés par `AWSClientFactory` à l'aide de `boto3`. Lorsque la cible est LocalStack, le code configure des credentials de test et redirige les clients vers `http://localhost:4566`. Les modules spécialisés collectent ensuite les données IAM, S3 et EC2.

Dans Azure, `AzureApi` encapsule les requêtes HTTP vers Topaz. L'objet gère le token local, les paramètres `api-version`, la pagination et les appels spécifiques au data plane Storage. Les modules de Discovery utilisent cet objet commun au lieu de réimplémenter la connexion réseau.

Le résultat de Discovery est un inventaire structuré. Il contient les ressources, leurs propriétés, des métadonnées de scan et, lorsque cela est pertinent, les erreurs de découverte. Cette approche permet au moteur Analysis de fonctionner uniquement sur des objets Python sans connaître les détails de transport HTTP.

2.6 Security Analysis

La phase **Security Analysis** applique des règles déterministes aux données collectées. Une règle ne se contente pas de produire un booléen. Lorsqu'un problème est détecté, elle génère un finding enrichi avec les informations nécessaires pour comprendre et traiter le risque.

Un finding contient typiquement :

Table 2.4 – Structure d'un finding de sécurité

Champ	Rôle
`rule_id`	Identifiant stable de la règle
`category`	Famille du risque
`resource` / `resource_id`	Ressource concernée

Champ	Rôle
`title`	Résumé lisible du problème
`description`	Explication de la condition détectée
`evidence`	Données ayant déclenché la règle
`recommendation`	Action de remédiation proposée
`risk_input` ou `risk_factors`	Variables utilisées par le scoring

Cette structure évite de mélanger détection et présentation. Le module Reporting peut afficher les mêmes findings en HTML ou en JSON sans connaître la logique ayant déclenché la règle.

Les règles AWS sont principalement regroupées dans `analysis/iam.py`, `analysis/s3.py` et `analysis/orphan.py`. Les règles Azure sont regroupées dans `analysis/rbac.py`, `analysis/storage.py`, `analysis/network.py` et `analysis/orphan.py`.

2.7 Risk Scoring

Le scoring transforme une liste de findings en une liste priorisée. Le but n'est pas uniquement d'étiqueter un problème comme « dangereux », mais d'aider à déterminer l'ordre de traitement.

Les deux implémentations utilisent volontairement des modèles différents. Le moteur AWS fonctionne sur une échelle de **0 à 10** à partir de l'impact, de la vraisemblance, de l'exposition et de certains tags. Le moteur Azure fonctionne sur une échelle de **0 à 100** : une base est associée à la sévérité initiale, puis plusieurs facteurs de risque sont ajoutés.

Cette différence ne bloque pas l'usage du rapport, car les deux moteurs produisent finalement les mêmes niveaux fonctionnels : `Low`, `Medium`, `High` et `Critical`. Elle illustre aussi que le pipeline est indépendant de la méthode mathématique retenue : le module Analysis fournit les paramètres et le module Scoring applique le modèle défini pour le provider.

2.8 Reporting

Le module Reporting constitue la dernière étape du pipeline. Il crée le dossier `output` si nécessaire et y sauvegarde plusieurs niveaux d'information.

Les deux projets génèrent :

— `inventory.json` : inventaire brut issu de Discovery ;
— `findings.json` : résultat de l'analyse avant présentation finale ;
— `report.json` : payload de rapport contenant le résumé et les findings scorés ;

— `report.html` : rapport visuel destiné à la consultation humaine.

Dans Azure, la création du laboratoire produit en plus `lab-manifest.json`, qui documente les ressources créées et les cas de sécurité attendus. Ce fichier est particulièrement utile pendant la validation, car il permet de comparer ce que le laboratoire contient avec ce que le scan détecte.

Le format JSON est utile pour l'intégration, l'automatisation ou le débogage. Le format HTML donne une vue synthétique : métadonnées, compteurs, sévérités, ressources découvertes, détails des findings, preuves et recommandations.

Chapitre 3

Outils et technologies utilisés

3.1 Python

Python constitue le langage principal des deux implémentations. Il porte la logique de découverte, d'analyse, de scoring, de sérialisation et de reporting. Son écosystème permet d'utiliser des SDK et bibliothèques adaptés aux deux environnements tout en conservant une architecture de code similaire [4].

3.2 Docker et Docker Compose

Docker fournit l'isolation nécessaire aux environnements Cloud fictifs. Les fichiers Compose décrivent les services, ports, volumes et paramètres utilisés par LocalStack et Topaz. Cette approche facilite la reconstruction du laboratoire et rend les scénarios de test reproductibles [5].

3.3 AWS, LocalStack et boto3

Côté AWS, **LocalStack** émule localement les services nécessaires au projet, notamment IAM, S3, STS et EC2. Le code communique avec ces services par **boto3**, le SDK AWS pour Python, en redirigeant les clients vers l'endpoint local. Cette combinaison permet de conserver une logique d'appels proche de celle d'un environnement AWS réel tout en travaillant dans un laboratoire contrôlé [7, 6].

Les règles de sécurité s'appuient notamment sur les concepts IAM et sur les mécanismes S3 de contrôle de l'accès public. La documentation AWS sert ainsi de référence fonctionnelle pour l'interprétation des policies, des identités et des paramètres de Block Public Access [8, 9].

3.4 Azure, Topaz, topaz-sdk et Requests

Côté Azure, **Topaz** fournit un émulateur local exposant des API compatibles avec les besoins du projet. Le package `topaz-sdk` est utilisé pour obtenir les informations d'authentification locales, tandis que la bibliothèque `requests` centralise les appels HTTP vers les API Topaz [10, 11].

La découverte reproduit une logique proche des API Azure Resource Manager : les ressources sont interrogées via le control plane, tandis que certaines opérations Storage utilisent le data plane. Les contrôles RBAC prennent en compte le rôle, le principal et le scope de l'affectation [12, 13]. Les règles Storage sont également alignées avec le fonctionnement d'`allowBlobPublicAccess` et des niveaux d'accès anonyme des containers [14].

3.5 PowerShell et automatisation locale

Des scripts **PowerShell** assurent l'orchestration des opérations courantes sous Windows : préparation de l'environnement, démarrage des conteneurs, initialisation du laboratoire, exécution du scan, réinitialisation et arrêt. Ils simplifient la démonstration sans remplacer le cœur applicatif Python.

3.6 Formats JSON et HTML

Le format **JSON** est utilisé pour conserver un résultat structuré et facilement réutilisable par un programme. Le format **HTML** sert à produire une vue lisible des findings, des scores, des preuves et des recommandations. Ces deux sorties répondent ainsi à deux besoins complémentaires : intégration technique et consultation humaine.

Chapitre 4

Développement de l'implémentation AWS

4.1 Architecture du projet

La version AWS conserve une structure Python relativement compacte. Les composants Cloud, les règles et la génération de rapport sont séparés dans des répertoires dédiés.

```
cloud-risk-tool
+---localstack
+---scripts
\---src
    \---cloud_risk_tool
        +---analysis
        +---discovery
        +---reporting
        \---scoring
```

Les responsabilités des fichier .py et des dossiers peuvent être résumées ainsi :

Table 4.1 – Principaux composants de l'implémentation AWS

Élément	Responsabilité principale
`aws.py`	Création des sessions et clients `boto3`, configuration de l'endpoint et normalisation des erreurs AWS
`config.py`	Paramètres de cible, région, endpoint, profile AWS, timeout et dossier de sortie
`cli.py`	Commande `scan`, options de ligne de commande et affichage du résumé final
`pipeline.py`	Orchestration Discovery -> Analysis -> Scoring -> Reporting

Élément	Responsabilité principale
`serialization.py`	Normalisation et sérialisation des données avant sauvegarde
`discovery/`	Inventaire IAM, S3 et EC2
`analysis/`	Règles de détection IAM, S3 et assets orphelins
`scoring/`	Calcul du score sur 10 et de la sévérité
`reporting/`	Génération des fichiers JSON et du rapport HTML
`localstack/`	Définition et initialisation du laboratoire AWS fictif
`scripts/`	Automatisation PowerShell des opérations courantes

Le fichier `pyproject.toml` définit le package `cloud-risk-tool`, exige Python 3.10 ou plus et déclare `boto3` comme dépendance principale. Il expose également un entry point CLI vers `cloud_risk_tool.cli:main`.

4.2 Discovery

La découverte AWS repose sur `boto3`. L'objet `AWSClientFactory` crée une session commune et fournit les clients nécessaires. Lorsque `target == "localstack"`, des credentials de test sont injectés et tous les clients utilisent l'URL définie dans `endpoint_url`.

Un extrait représentatif de cette logique est le suivant :

```
if self.settings.target == "localstack":
    session_kwargs.update(
        aws_access_key_id="test",
        aws_secret_access_key="test",
        aws_session_token="test",
    )
...
if self.settings.endpoint_url:
    kwargs["endpoint_url"] = self.settings.endpoint_url
return self.session.client(**kwargs)
```

La fabrique configure aussi les retries et les timeouts Botocore. Cette centralisation évite à `iam.py`, `s3.py` et `ec2.py` de connaître les détails de connexion.

Le `DiscoveryManager` commence par tenter un appel STS `GetCallerIdentity`, résout ensuite la liste des régions puis lance successivement `IAMDiscovery`, `S3Discovery` et `EC2Discovery`. Il ajoute enfin un résumé de l'inventaire et centralise les erreurs de

découverte.

```
AWSClientFactory
|-- STS : identité
|-- IAMDiscovery -> Users / Roles / Groups / Policies
|-- S3Discovery  -> Buckets / Policy / ACL / Public Access Block
`-- EC2Discovery -> Instances / Volumes / Elastic IP / ENI
                      |
                  Inventory
```

4.2.1 Découverte IAM

`discovery/iam.py` cherche d'abord à utiliser `GetAccountAuthorizationDetails`, qui permet de récupérer un ensemble riche d'informations sur les utilisateurs, rôles, groupes et policies. Si cette opération n'est pas disponible ou échoue, le code possède un mécanisme de fallback qui utilise les opérations de liste plus classiques.

Pour chaque utilisateur, les données normalisées comprennent notamment :

- le nom, l'ARN, le chemin et la date de création ;
- les groupes ;
- les policies inline et les policies managées attachées ;
- les permissions boundaries ;
- les tags ;
- les clés d'accès et leur date de création ;
- les dispositifs MFA ;
- l'existence éventuelle d'un login profile console.

Pour un rôle, le module ajoute la trust policy `AssumeRolePolicyDocument`, élément indispensable pour détecter un rôle assumable par un principal non restreint.

Les managed policies référencées sont également enrichies afin que le moteur Analysis dispose du document de policy réel, et pas uniquement de son ARN. Cette étape est importante pour l'analyse de permissions : la détection doit lire `Action`, `Resource` et `Effect`.

4.2.2 Découverte S3

`discovery/s3.py` commence par `ListBuckets`, puis interroge chaque bucket individuellement. Pour chaque ressource, l'inventaire inclut :

- le nom et l'ARN ;
- la région ;
- la configuration `PublicAccessBlock` ;
- le résultat de `GetBucketPolicyStatus` ;
- la bucket policy ;

— l'ACL ;
— les tags.

Le module tente également de récupérer le Block Public Access au niveau du compte lorsque l'identifiant du compte est disponible. La configuration effective peut ainsi être déterminée plus tard en combinant protections du bucket et protections du compte.

4.2.3 Découverte EC2

`discovery/ec2.py` collecte quatre ensembles utiles au projet : instances, volumes, Elastic IP et interfaces réseau. Les instances permettent de documenter l'environnement ; les trois autres collections sont essentielles à l'identification des assets orphelins.

Pour les volumes, le module conserve notamment l'état, la taille, le type, la zone de disponibilité et surtout la liste `Attachments`. Pour les Elastic IP, il conserve les identifiants d'association, d'instance et d'interface réseau. Pour les ENI, il collecte le statut, le type, l'indicateur `RequesterManaged` et l'objet `Attachment`.

La logique de détection peut ensuite travailler uniquement sur ces champs, sans effectuer de nouvel appel Cloud.

4.3 Détection des mauvaises configurations IAM

L'analyse IAM est réalisée par `IAMAnalyzer`. Elle traite les policies directes d'un utilisateur ou d'un rôle, mais aussi les permissions héritées par l'intermédiaire des groupes d'un utilisateur.

Le module applique six règles :

Table 4.2 – Règles de détection des mauvaises configurations IAM

Règle	Condition détectée	Impact / Likelihood / Exposure
`IAM-001`	`Action: "*"` et `Resource: "*"` dans une instruction `Allow`	5 / 5 / 1
`IAM-002`	Policy AWS `AdministratorAccess` attachée	5 / 5 / 1
`IAM-003`	Action sensible pouvant faciliter une élévation de privilèges sur `Resource: "*"`	5 / 4 / 0

Règle	Condition détectée	Impact / Likelihood / Exposure
IAM-004	Trust policy autorisant un principal wildcard sans condition restrictive	5 / 4 / 1
IAM-005	Utilisateur avec mot de passe console mais sans MFA	4 / 4 / 0
IAM-006	Clé d'accès active plus ancienne que le seuil configuré, 90 jours par défaut	4 / 3 / 0

IAM-003 ne cherche pas toutes les actions AWS. Il utilise une liste ciblée d'actions sensibles telles que iam:PassRole, iam:CreatePolicyVersion, iam:AttachUserPolicy, iam:PutRolePolicy, iam:CreateAccessKey, sts:AssumeRole ou encore certaines opérations Lambda. Une action sensible ne déclenche la règle que lorsqu'elle est combinée à une ressource wildcard.

La règle IAM-004 illustre le principe de traitement des policies. Le code parcourt les statements de la trust policy et vérifie trois conditions : Effect doit être Allow, le principal doit être public et aucune Condition ne doit réduire la portée de l'accès.

```
if (
    str(statement.get("Effect", "")).lower() == "allow"
    and principal_is_public(statement.get("Principal"))
    and not statement.get("Condition")
):
    # création du finding IAM-004
```

Chaque finding est construit avec make_finding(), qui ajoute le rule_id, la ressource, l'évidence, la recommandation et les entrées de scoring. La fonction de déduplication empêche la production répétée du même couple rule_id/resource_id.

Dans le laboratoire de référence, trois cas IAM sont volontairement présents : IAM-001, IAM-004 et IAM-005. Les règles IAM-002, IAM-003 et IAM-006 restent cependant implémentées et peuvent être déclenchées si l'inventaire contient les configurations correspondantes.

4.4 Détection des buckets S3 exposés

Le module S3Analyzer combine plusieurs sources avant de décider qu'un bucket est exposé. Il ne se contente donc pas de vérifier une seule propriété.

La première étape consiste à construire la configuration Block Public Access effective à partir des valeurs du bucket et, lorsqu'elle existe, de la configuration au niveau du compte. Les quatre protections prises en compte sont :

— `BlockPublicAcls` ;
— `IgnorePublicAcls` ;
— `BlockPublicPolicy` ;
— `RestrictPublicBuckets`.

Le moteur inspecte ensuite les ACL et les bucket policies. Pour les ACL, les groupes globaux `AllUsers` et `AuthenticatedUsers` sont considérés comme publics. Pour les policies, une instruction est considérée publique lorsque son effet est `Allow`, que son principal est public et qu'aucune condition ne limite ce principal.

Les actions publiques sont séparées entre opérations de lecture et opérations d'écriture. Cette distinction permet de produire un niveau de risque différent.

Table 4.3 – Règles de détection des buckets S3 exposés

Règle	Condition	Paramètres de risque
`S3-001`	Écriture publique effective sur le bucket	impact 5, likelihood 5, exposure 1
`S3-002`	Lecture publique effective sur le bucket	impact 4, likelihood 5, exposure 1
`S3-003`	Une ou plusieurs protections Block Public Access effectives sont désactivées	impact 3, likelihood 3, exposure 0

Le code donne la priorité à l'écriture : si `public_write` est vrai, `S3-001` est créé. Sinon, si le bucket est publiquement lisible, `S3-002` est créé. `S3-003` est évaluée indépendamment ; un bucket exposé peut donc produire à la fois un finding d'exposition et un finding de hardening.

Dans le laboratoire, `crl-public-upload` déclenche la règle d'écriture publique, tandis que `crl-public-documents` déclenche la règle de lecture publique. Les deux ont un Block Public Access incomplet et déclenchent donc également `S3-003`. `crl-private-data` sert de cas négatif : il est découvert mais ne doit pas générer de finding S3.

4.5 Détection des assets orphelins

L'analyse des assets orphelins se trouve dans `analysis/orphan.py`. Les règles sont volontairement simples et explicables : une ressource est considérée comme potentiel-

lement orpheline lorsqu'elle est dans un état disponible mais n'est rattachée à aucun consommateur identifié.

Table 4.4 – Règles de détection des assets orphelins AWS

Règle	Ressource	Critère
`ORPHAN-001`	EBS Volume	`state == available` et liste d'attachments vide
`ORPHAN-002`	Elastic IP	absence d'`association_id`, `instance_id` et `network_interface_id`
`ORPHAN-003`	Network Interface	interface customer-managed, état `available`, sans attachment

Certaines précautions réduisent les faux positifs. Pour les ENI, le moteur ignore les interfaces marquées `RequesterManaged` et les types d'interface qui ne correspondent pas à une interface réseau standard. Une ressource détectée est qualifiée de « potentially orphaned » lorsque la seule observation technique ne suffit pas à prouver qu'elle est inutilisable ; la recommandation demande donc une validation avant suppression.

Cette approche est particulièrement adaptée au contrôle des ressources inutilisées : le scanner identifie les candidates, puis l'équipe Cloud peut confirmer le contexte métier ou opérationnel.

4.6 Risk Scoring

Le moteur AWS `RiskScoringEngine` calcule un score compris entre `0.0` et `10.0`. Les valeurs d'entrée `impact` et `likelihood` sont bornées entre 1 et 5, et `exposure` entre 0 et 1.

Le score de base donne plus de poids à l'impact qu'à la vraisemblance :

```
base = (((impact × 0.6) + (likelihood × 0.4)) / 5) × 9
score = min(10, base + exposure_bonus + tag_bonus)
```

Le point restant sur l'échelle de dix permet de valoriser une exposition directe. Des tags peuvent ensuite ajouter jusqu'à `1.5` point :

Table 4.5 – Bonus contextuels appliqués au scoring AWS

Facteur de contexte	Bonus
`Criticality = high/critical`	+0.7

Facteur de contexte	Bonus
`DataClassification = confidential/restricted/secret`	+0.8
`Environment = prod/production`	+0.4
Bonus total maximum	1.5

Le score final est plafonné à 10. La sévérité est déterminée selon les seuils suivants :

Table 4.6 – Seuils de sévérité du scoring AWS

Score	Sévérité
`9.0 - 10.0`	Critical
`7.0 - 8.9`	High
`4.0 - 6.9`	Medium
`< 4.0`	Low

Ce modèle permet à deux findings issus de la même règle d'obtenir une priorité différente selon leur contexte. Un bucket public contenant des données confidentielles peut ainsi atteindre un score plus élevé qu'un bucket de test présentant la même configuration technique.

Le résultat scoré contient aussi `score_breakdown`, qui indique les valeurs utilisées, les bonus et la formule. Le rapport peut donc expliquer pourquoi une sévérité a été attribuée au lieu de présenter un score opaque.

4.7 Exécution

Le répertoire `scripts/` fournit une interface opérationnelle simple pour un poste Windows utilisant PowerShell. Ces scripts ne remplacent pas le code Python ; ils automatisent les étapes répétitives nécessaires au laboratoire.

Table 4.7 – Scripts PowerShell de l'implémentation AWS

Script	Fonction
`quickstart.ps1`	Prépare le venv, installe le package, démarre le laboratoire et lance le scan
`start-lab.ps1`	Construit/démarre LocalStack et attend la fin de l'initialisation
`scan-localstack.ps1`	Relance uniquement le scan sur le laboratoire existant
`reset-lab.ps1`	Supprime le laboratoire et ses volumes puis le recrée
`stop-lab.ps1`	Arrête et supprime le service Docker actif

Avant la première utilisation dans une session PowerShell où l'exécution des scripts est restreinte, la politique peut être contournée uniquement pour le processus courant :

```
Set-ExecutionPolicy -Scope Process Bypass
```

Le scénario complet s'exécute ensuite avec :

```
.\scripts\quickstart.ps1
```

Le script vérifie la présence de Docker et de Python, crée un environnement virtuel sous `%LOCALAPPDATA%\CloudRiskTool\venv`, installe le projet en mode editable, exécute `start-lab.ps1`, puis `scan-localstack.ps1`.

Le scan seul appelle finalement :

```
python -m cloud_risk_tool scan --target localstack --output-dir output
```

La possibilité de séparer démarrage et scan est utile pendant le développement d'une règle. Le laboratoire peut rester inchangé pendant que le code Analysis ou Reporting évolue, puis le scan est relancé sans reconstruire toutes les ressources.

4.8 Reporting

Le générateur AWS produit quatre fichiers dans `output/` :

```
output/
|-- inventory.json
|-- findings.json
|-- report.json
`-- report.html
```

`inventory.json` conserve les métadonnées de scan, l'inventaire IAM/S3/EC2 et les erreurs éventuelles. `findings.json` sauvegarde le résultat de l'analyse. `report.json` combine un résumé de l'inventaire avec les findings scorés. Enfin, `report.html` fournit l'affichage visuel.

Le HTML présente notamment :

- les informations de scan pertinentes ;
- le nombre de findings par sévérité ;
- un résumé de l'inventaire ;
- chaque finding avec règle, ressource, score et sévérité ;
- les preuves techniques ;
- la recommandation de remédiation ;
- les erreurs de scan lorsqu'elles existent.

Le générateur masque les identifiants de compte factices composés uniquement de zéros afin d'éviter de présenter une information LocalStack sans valeur comme s'il s'agissait d'un identifiant AWS réel.

Le rapport principal peut être ouvert directement depuis PowerShell :

```
Start-Process .\output\report.html
```



Figure 4.1



Figure 4.2

Chapitre 5

Développement de l'implémentation Azure

5.1 Architecture du projet

La version Azure est plus riche en modules de Discovery parce qu'elle manipule davantage de familles de ressources.

```
azure-risk-tool
+---scripts
+---src
|   \---azure_risk_tool
|       +---analysis
|       +---discovery
|       +---reporting
|       \---scoring
\---topaz
```

Les composants principaux sont :

Table 5.1 – Principaux composants de l'implémentation Azure

Élément	Responsabilité principale
`azure.py`	Client HTTP Topaz, token local, pagination, gestion des erreurs et authentification Storage Shared Key
`config.py`	Endpoint ARM, endpoint Storage, subscription, location, TLS, timeout et output
`lab.py`	Construction du laboratoire et du `lab-manifest.json`

Élément	Responsabilité principale
`cli.py`	Commandes `scan`, `create-lab` et `lab-info`
`pipeline.py`	Enchaînement Discovery -> Analysis -> Scoring -> Reporting
`discovery/`	Collecte Identity, Resources, RBAC, Storage, Compute et Network
`analysis/`	Règles RBAC, Storage, Network et Orphaned Assets
`scoring/`	Score sur 100 à partir de la sévérité initiale et des facteurs de risque
`reporting/`	JSON et rapport HTML
`topaz/`	Définition du service Docker et point d'entrée de création du lab
`scripts/`	Automatisation des opérations depuis PowerShell

`pyproject.toml` exige Python 3.10 ou supérieur et utilise principalement `requests` et `topaz-sdk`. Le package expose également un entry point `azure-risk-tool` vers le CLI.

5.2 Discovery

5.2.1 Connexion aux API Topaz

La connexion est centralisée dans la classe `AzureApi`. À son initialisation, elle crée une `requests.Session`, définit le header d'acceptation et récupère un token grâce à `AzureLocalCredential` fourni par `topaz-sdk`.

Le mécanisme commun `request()` ajoute ensuite le token Bearer et, lorsque nécessaire, le paramètre `api-version` :

```
headers = {"Authorization": f"Bearer {self._token}"}
if body is not None:
    headers["Content-Type"] = "application/json"
response = self.session.request(
    method,
    url,
    params=query,
    json=body,
    headers=headers,
    timeout=self.settings.timeout,
    verify=self.settings.verify_tls,
```

```
)
```

La méthode `list_all()` gère les réponses paginées en suivant `nextLink` ou `@odata.nextLink`. La méthode `storage_request()` est séparée car l'accès Blob utilise un data plane différent et une authentification Shared Key. Le code calcule la signature HMAC nécessaire et envoie les requêtes sur le port `8891`.

Cette distinction reflète deux types d'accès utilisés par le projet : le **control plane** pour découvrir les ressources Azure et leurs propriétés, puis le **data plane Storage** pour lister les containers et lire leur niveau d'accès public.

5.2.2 Organisation de la découverte

`DiscoveryManager` crée une seule instance `AzureApi`, vérifie la disponibilité de Topaz puis lance les modules dans un ordre logique :

```
AzureApi -> Identity -> Resources -> RBAC -> Storage
-> Compute -> Network -> Inventory Azure -> Resource counts
```

L'ordre RBAC après les groupes de ressources est utile, car `discovery/rbac.py` construit les scopes à analyser à partir de la subscription et des Resource Groups découverts.

Les fonctions de `discovery/common.py` ajoutent notamment `resource_group` et `discovery_source` aux ressources normalisées. Ces métadonnées simplifient le reporting et l'analyse sans modifier les objets bruts plus que nécessaire.

5.2.3 Identity et RBAC

`identity.py` récupère deux collections :

- `/v1.0/users` ;
- `/v1.0/servicePrincipals`.

`rbac.py` construit ensuite la liste des scopes. Il interroge les Role Assignments au niveau subscription puis au niveau de chacun des Resource Groups. Un ensemble `seen` évite les doublons.

Le module tente aussi de lister les Role Definitions. L'analyse n'en dépend toutefois pas obligatoirement pour les rôles built-in ciblés, car les identifiants GUID de `Owner`, `Contributor` et `Reader` sont connus dans `analysis/rbac.py`.

5.2.4 Storage

`storage.py` découvre d'abord les Storage Accounts avec l'API de management. Pour chaque compte, il appelle `listKeys`, obtient une clé locale puis utilise le data plane Blob afin de lister les containers. Le niveau public est lu dans le header `x-ms-blob-public-access`.

Le résultat distingue donc deux niveaux de configuration :

— le Storage Account, avec des propriétés comme `allowBlobPublicAccess` et `publicNetworkAccess` ;
— le Blob Container, avec son niveau `private`, `blob` ou `container`.

Cette distinction est indispensable : autoriser techniquement la création de containers publics au niveau du compte ne signifie pas qu'un container est déjà exposé, et inversement le finding sur un container doit être lié à sa configuration effective.

5.2.5 Compute et Network

`compute.py` récupère les Virtual Machines et Managed Disks. `network.py` récupère Virtual Networks, Network Security Groups, Network Interfaces et Public IP Addresses.

Ces collections sont nécessaires pour reconstituer les relations entre ressources. L'analyse des assets orphelins ne repose pas sur un simple nom : elle compare les IDs référencés par les VM, les NIC et les Public IP. Cette méthode est plus fiable qu'une convention de nommage.

Les NSG sont également collectés afin d'appliquer deux contrôles réseau complémentaires. Même si le sujet principal est centré sur IAM/RBAC, stockage exposé et assets orphelins, les données réseau disponibles rendent possible la détection d'une exposition directe des services d'administration.

5.3 Détection des mauvaises configurations RBAC

`analysis/rbac.py` crée d'abord un index des utilisateurs et service principals à partir de leur identifiant. Il parcourt ensuite chaque Role Assignment et extrait trois éléments : le rôle, le scope et le principal.

Trois règles sont implémentées :

Table 5.2 – Règles de détection des mauvaises configurations RBAC

Règle	Condition	Sévérité initiale	Facteurs principaux
`AZ-RBAC-001`	Rôle Owner au scope exact de la subscription	Critical	privilege 10, exploitability 8
`AZ-RBAC-002`	Rôle Contributor au scope exact de la subscription	High	privilege 8, exploitability 7

Règle	Condition	Sévérité initiale	Facteurs principaux
AZ-RBAC-003	Role Assignment vers un principal absent de la découverte Identity	Medium	privilege 4

Le contrôle du scope est volontairement strict. `_is_subscription_scope()` vérifie que l'identifiant normalisé contient seulement les deux segments `subscriptions/<id>`. Ainsi, un rôle Contributor sur un seul Resource Group n'est pas traité de la même manière qu'un Contributor couvrant toute la subscription.

Le laboratoire déclenche `AZ-RBAC-001` pour Alice Admin et `AZ-RBAC-002` pour `sp-deployment`. Les affectations Reader au niveau `rg-production` jouent le rôle de cas normaux. `AZ-RBAC-003` n'est pas introduite volontairement dans le lab, mais elle permet de détecter une affectation potentiellement stale lorsqu'un principal référencé n'apparaît plus dans l'inventaire des identités.

5.4 Détection du stockage exposé

Deux règles composent l'analyse Storage.

`AZ-STG-001` est déclenchée lorsque la propriété du Storage Account `allowBlobPublicAccess` vaut `true`. Cette configuration ne prouve pas à elle seule qu'un blob est actuellement public ; elle signifie que le compte permet à un container d'activer l'accès anonyme. La règle est donc classée `Medium` initialement et recommande de désactiver cette capacité lorsqu'elle n'est pas nécessaire.

`AZ-STG-002` examine le niveau d'accès découvert sur chaque Blob Container. Une valeur `blob` ou `container` est traitée comme exposition anonyme. Cette règle est plus directe et commence en sévérité `High`.

Table 5.3 – Règles de détection du stockage Azure exposé

Règle	Ressource	Condition	Sévérité initiale
AZ-STG-001	Storage Account	`allowBlobPublicAccess == true`	Medium
AZ-STG-002	Blob Container	`public_access` égal à `blob` ou `container`	High

Le moteur tient aussi compte du tag `data_classification`. Une valeur `confidential`, `restricted` ou `sensitive` donne un facteur `sensitivity` de 8 ; dans le cas contraire, le

facteur vaut 3. Le même problème technique obtient donc une priorité plus importante lorsque le stockage contient des données classifiées.

Dans le laboratoire, `stpublicdata` et son container `public-data` déclenchent les deux règles. `stprivatebackups` sert de cas normal.

5.4.1 Contrôles réseau complémentaires

Deux contrôles supplémentaires sont implémentés dans `analysis/network.py` et apparaissent dans les résultats de la version Azure.

`AZ-NET-001` recherche les règles NSG `Inbound` + `Allow` dont la source est publique (`*`, `0.0.0.0/0`, `Internet` ou `Any`) et dont le port cible correspond à un service d'administration. Les ports suivis sont SSH `22`, RDP `3389`, WinRM `5985` et WinRM HTTPS `5986`. Les ranges sont également interprétés : une plage contenant l'un de ces ports est considérée comme exposante.

`AZ-NET-002` identifie une Network Interface associée à une Public IP découverte. Ce contrôle ne prétend pas qu'une NIC publique est automatiquement vulnérable ; il produit un finding `Medium` qui demande de confirmer la nécessité de l'exposition et l'efficacité des règles NSG.

Dans le laboratoire, deux règles NSG distinctes exposent SSH et RDP. Le moteur produit donc deux findings `AZ-NET-001`, auxquels s'ajoute `AZ-NET-002` pour la NIC de la VM possédant une Public IP.

5.5 Détection des assets orphelins

Le module Azure reconstruit d'abord les dépendances avant de signaler une ressource. Il collecte les IDs et noms de disques référencés par les VM, les IDs de NIC des `networkProfiles` et, à partir des NIC réellement attachées, les Public IP utilisées.

Trois règles sont ensuite appliquées :

Table 5.4 – Règles de détection des assets orphelins Azure

Règle	Ressource	Condition principale
`AZ-ORPH-001`	Managed Disk	Non référencé par une VM et absence de `managedBy`
`AZ-ORPH-002`	Network Interface	Non référencée par une VM et absence de propriété `virtualMachine`
`AZ-ORPH-003`	Public IP	Non référencée par une NIC attachée et absence d'`ipConfiguration`

Le contrôle des disques accepte à la fois les références par ID et par nom, notamment pour l'OS disk dont la représentation peut ne pas contenir un ID de Managed Disk identique à celui d'un data disk explicite.

Le disque `disk-old-backup`, la NIC `nic-unused` et la Public IP `pip-unused` sont créés spécifiquement pour tester les trois règles. Le disque orphelin porte en plus `data_classification=confidential`, ce qui augmente sa sensibilité dans le scoring.

5.6 Risk Scoring

Le modèle Azure est distinct du modèle AWS. Chaque finding reçoit d'abord une `initial_severity`, puis un score de base :

Table 5.5 – Scores de base du modèle de scoring Azure

Sévérité initiale	Base
Low	15
Medium	40
High	65
Critical	85

À cette base sont ajoutés jusqu'à quatre facteurs, chacun borné entre 0 et 10 :

- `internet_exposure` ;
- `privilege` ;
- `sensitivity` ;
- `exploitability`.

La formule est volontairement simple :

```
risk_score = min(100, base(initial_severity) + somme(risk_factors))
```

Les seuils finaux sont :

Table 5.6 – Seuils de sévérité du scoring Azure

Score	Sévérité finale
85 - 100	Critical
60 - 84	High
30 - 59	Medium
< 30	Low

L'intérêt du modèle est de conserver l'intuition de la règle initiale tout en la modifiant avec le contexte. Un Blob Container public commence par exemple à 65 (`High`), mais l'exposition Internet, la sensibilité et l'exploitabilité peuvent l'amener en `Critical`.

À l'inverse, une NIC publique commence à `Medium` et reçoit un facteur d'exposition et d'exploitabilité plus faible, ce qui la maintient généralement dans la même catégorie. Le modèle évite ainsi de traiter une simple surface d'exposition comme un accès anonyme direct à des données sensibles.

5.7 Exécution

Les scripts PowerShell Azure visent le même objectif que dans AWS, avec une différence : le laboratoire Topaz est initialisé par le package Python lui-même.

Table 5.7 – Scripts PowerShell de l'implémentation Azure

Script	Fonction
`quickstart.ps1`	Prépare `.venv`, installe le package, lance Topaz, crée le lab, effectue le scan et ouvre le rapport
`start-lab.ps1`	Lance uniquement le conteneur Topaz
`scan-topaz.ps1`	Installe/actualise le package et relance le scan
`reset-lab.ps1`	Supprime le volume Topaz, redémarre le service et recrée les ressources
`stop-lab.ps1`	Arrête le conteneur sans supprimer le volume

Après la préparation réseau locale et l'autorisation temporaire des scripts :

```
Set-ExecutionPolicy -Scope Process Bypass
```

le scénario complet est :

```
.\scripts\quickstart.ps1
```

Ce script réalise les actions suivantes :

1. vérifie que Docker fonctionne ;
2. crée `.venv` si nécessaire ;
3. installe le projet avec `pip install -e .` ;
4. exécute `docker compose -f .\topaz\compose.yml up -d --build` ;
5. appelle `topaz/init-azure.py` pour créer le laboratoire ;
6. exécute `python -m azure_risk_tool scan --output .\output` ;
7. ouvre `output/report.html` s'il existe.

Le CLI fournit également les commandes `create-lab` et `lab-info`. Cette dernière lit `output/lab-manifest.json` et affiche l'état de référence sans reconstruire le laboratoire.

5.8 Reporting

Le reporting Azure génère les mêmes quatre fichiers principaux que la version AWS :

```
output/
|-- inventory.json
|-- findings.json
|-- report.json
|-- report.html
`-- lab-manifest.json   # créé lors de l'initialisation du laboratoire
```

Le payload final contient le provider, la subscription, les compteurs de ressources, le nombre de findings, les sévérités et le score maximum. Le rapport HTML affiche ensuite ces données sous une forme visuelle.

Le générateur sait reconnaître la subscription fictive utilisée par Topaz (`11111111-...`) et peut la masquer dans la présentation afin qu'un identifiant de laboratoire ne soit pas confondu avec un véritable identifiant d'abonnement Azure.

Les findings affichent la ressource, la règle, la sévérité, le score, la description, l'évidence et la recommandation. Le rapport sert donc à la fois de synthèse et de support de remédiation.

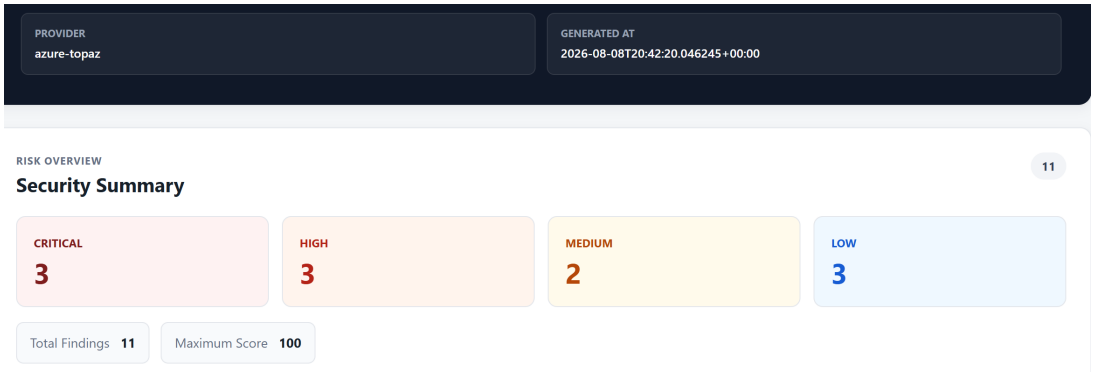


Figure 5.1

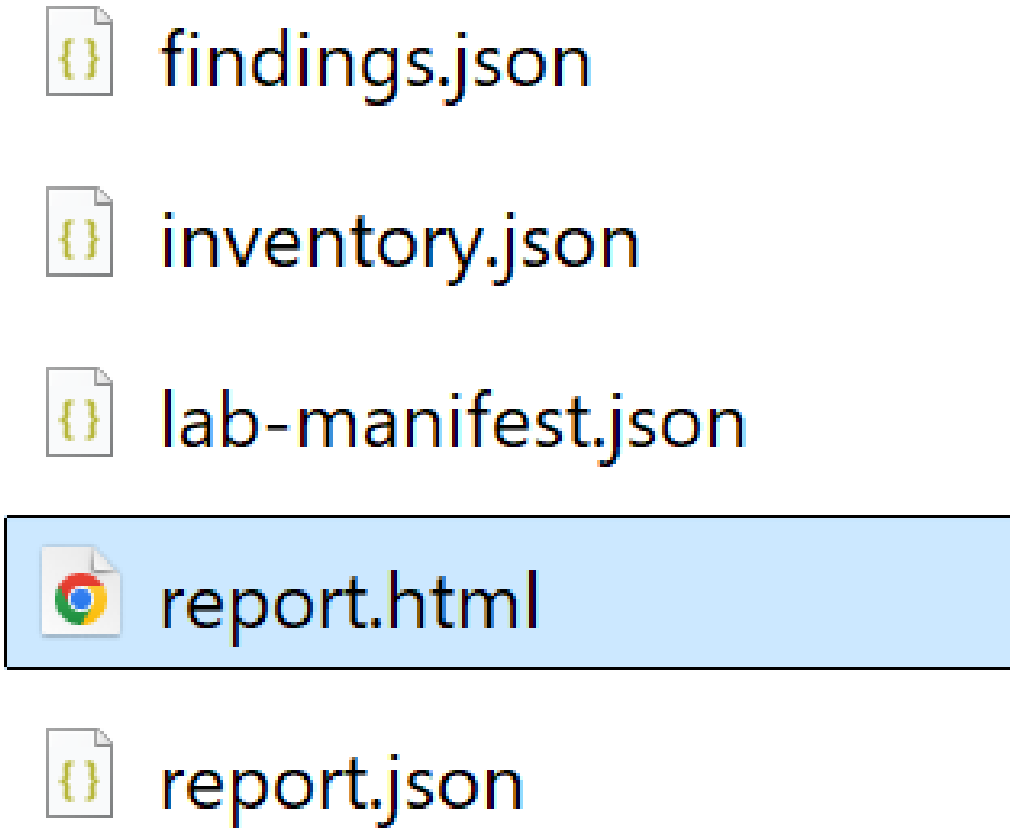

findings.json
inventory.json
lab-manifest.json
report.html
report.json

Figure 5.2

Chapitre 6

Tests, résultats et constats

6.1 Scénarios de test

La validation du projet repose sur des laboratoires contrôlés plutôt que sur des environnements Cloud aléatoires. Cette stratégie permet de connaître à l'avance les ressources et les configurations introduites, puis de vérifier si le pipeline retrouve effectivement les risques prévus.

Le principe de test est le suivant :

```
Création du laboratoire -> Configurations sûres + vulnérables
-> Discovery -> Inventaire -> Analysis -> Findings -> Scoring
-> Comparaison avec les cas attendus
```

Les tests sont donc des tests fonctionnels de bout en bout. Ils valident simultanément plusieurs éléments : accès au laboratoire, capacité de découverte, qualité des données collectées, déclenchement des règles, calcul du score et génération des fichiers de sortie.

6.1.1 Scénario AWS

Le scénario de référence commence par la reconstruction de LocalStack. L'utilisation de `reset-lab.ps1` permet de supprimer l'état courant et de repartir d'un environnement propre :

```
.\scripts\reset-lab.ps1
.\scripts\scan-localstack.ps1
```

Il est également possible d'utiliser directement `quickstart.ps1` lors d'une première exécution.

Le laboratoire contient des contrôles positifs et négatifs :

Table 6.1 – Scénarios fonctionnels de test AWS

Domaine	Cas volontairement vulnérable	Cas de contrôle
IAM	wildcard permissions, rôle public, console sans MFA	utilisateur avec policy S3 limitée
S3	lecture publique, écriture publique, BPA désactivé	bucket privé avec BPA complet
Assets	volume, EIP et ENI non attachés	volume et EIP réellement associés

Les règles `IAM-002`, `IAM-003` et `IAM-006` ne font pas partie du scénario minimal créé par `init-aws.sh`, mais elles restent couvertes par le moteur. Elles peuvent être testées séparément en ajoutant une policy `AdministratorAccess`, une action sensible avec ressource wildcard ou une ancienne access key active.

6.1.2 Scénario Azure

Pour Azure, le reset supprime le volume Topaz, redémarre le service et recrée les ressources :

```
.\scripts\reset-lab.ps1
.\scripts\scan-topaz.ps1
```

`lab.py` crée explicitement une matrice de cas risqués et de ressources normales :

Table 6.2 – Scénarios fonctionnels de test Azure

Domaine	Cas volontairement vulnérable	Cas de contrôle
RBAC	Owner et Contributor au scope subscription	Reader au scope Resource Group
Storage	compte autorisant le public + container public	compte et container privés
Network	SSH/RDP depuis Internet + NIC avec Public IP	NSG interne
Assets	disque, NIC et Public IP non attachés	disque/NIC/Public IP utilisés par la VM

Le fichier `lab-manifest.json` joue le rôle de référence technique. Il documente les ressources créées et permet de vérifier les cas attendus indépendamment du rapport du scanner.

La règle `AZ-RBAC-003` n'est pas déclenchée dans le laboratoire normal parce que tous les principals utilisés par les Role Assignments existent. Cette règle peut être validée en

créant ou en conservant une affectation pointant vers un principal absent de l'inventaire Identity.

6.2 Résultats AWS

Les résultats de référence ci-dessous sont déterminés par les ressources créées dans `init-aws.sh`, les règles de la version finale du code et la formule du `RiskScoringEngine`. Si le laboratoire est modifié, le nombre de findings peut naturellement évoluer.

Table 6.3 – Résultats de référence de l'implémentation AWS

Règle	Ressource de test	Score de référence	Sévérité
`IAM-001`	`crl-risky-admin`	10.0	Critical
`IAM-004`	`crl-public-trust-role`	9.3	Critical
`IAM-005`	`crl-console-no-mfa`	7.2	High
`S3-001`	`crl-public-upload`	10.0	Critical
`S3-002`	`crl-public-documents`	10.0	Critical
`S3-003`	`crl-public-upload`	6.1	Medium
`S3-003`	`crl-public-documents`	6.9	Medium
`ORPHAN-001`	`crl-orphan-volume`	4.3	Medium
`ORPHAN-002`	`crl-orphan-eip`	3.2	Low
`ORPHAN-003`	`crl-orphan-eni`	2.5	Low

Le résumé correspondant est donc :

Table 6.4 – Synthèse des résultats de référence AWS

Indicateur	Valeur de référence
Findings	10
Critical	4
High	1
Medium	3
Low	2
Score maximum	10.0

Les scores élevés des buckets s'expliquent en partie par les tags du laboratoire. `crl-public-documents` est marqué avec une criticité élevée et une classification confidentielle ; son finding de lecture publique obtient donc le bonus contextuel maximum autorisé par le moteur. Le finding `S3-003` sur le même bucket reste `Medium`, car son impact et sa likelihood de base sont plus faibles.

Le cas `crl-public-upload` confirme aussi que l'analyse distingue lecture et écriture : l'accès public en écriture produit `S3-001`, considéré plus dangereux qu'un défaut de hardening seul.

Du côté IAM, la policy `Action: "*" / Resource: "*"` produit le score maximal, tandis que l'absence de MFA reste `High`. Cette différence illustre la priorité donnée à la capacité d'action complète sur le compte par rapport à un défaut d'hygiène d'authentification, même si ce dernier demeure important.

Enfin, les assets orphelins obtiennent des niveaux plus modérés. Le but est de les rendre visibles sans les placer automatiquement au même niveau qu'une exposition publique de données ou une permission administrative complète.

6.3 Résultats Azure

Le laboratoire Azure produit dix findings de référence. Deux règles réseau sont déclenchées deux fois parce que le NSG contient deux règles d'administration publiques distinctes : une pour RDP et une pour SSH.

Table 6.5 – Résultats de référence de l'implémentation Azure

Règle	Ressource / cas	Score de référence	Sévérité
`AZ-RBAC-001`	Alice Admin — Owner subscription	100	Critical
`AZ-RBAC-002`	sp-deployment — Contributor subscription	80	High
`AZ-STG-001`	`stpublicdata`	56	Medium
`AZ-STG-002`	container `public-data`	91	Critical
`AZ-NET-001`	RDP 3389 depuis Internet	83	High
`AZ-NET-001`	SSH 22 depuis Internet	83	High
`AZ-NET-002`	NIC `nic-web-01` avec Public IP	49	Medium
`AZ-ORPH-001`	`disk-old-backup`	48	Medium
`AZ-ORPH-002`	`nic-unused`	15	Low
`AZ-ORPH-003`	`pip-unused`	17	Low

Le résumé de référence est :

Table 6.6 – Synthèse des résultats de référence Azure

Indicateur	Valeur de référence
Findings	10
Critical	2
High	3
Medium	3
Low	2
Score maximum	100

Indicateur	Valeur de référence

Le rôle Owner au niveau subscription atteint 100 parce que sa base `Critical` vaut déjà 85 et que les facteurs `privilege=10` et `exploitability=8` dépassent le plafond de 100. Le Contributor reste `High` avec un score de 80.

Le cas Storage montre l'effet du contexte. Le simple fait que `allowBlobPublicAccess` soit activé sur le compte produit un score de 56, donc `Medium`. En revanche, le container réellement accessible anonymement reçoit une base High à laquelle s'ajoutent exposition Internet, sensibilité et exploitabilité ; le résultat atteint 91 et devient `Critical`.

Les deux règles d'administration publique du NSG atteignent 83. Elles restent `High`, juste sous le seuil Critical de 85. Cela permet de prioriser le Blob public et l'Owner subscription, tout en conservant l'exposition SSH/RDP à un niveau élevé.

Les assets orphelins suivent la même logique que dans AWS. Le disque confidentiel est `Medium` car son facteur de sensibilité augmente le score. La NIC et la Public IP orphelines restent `Low` dans ce laboratoire.

6.4 Validation des détections

La validation ne doit pas se limiter au nombre global de findings. Un scanner pourrait produire dix résultats tout en associant les mauvaises règles aux mauvaises ressources. La vérification se fait donc au niveau de la correspondance **cas introduit -> ressource découverte -> règle déclenchée**.

6.4.1 Pour AWS, la matrice principale est :

Table 6.7 – Matrice de validation des détections AWS

Cas du laboratoire	Détection attendue	Validation logique
Wildcard IAM complet	`IAM-001`	Action et Resource wildcard présents
Trust policy publique	`IAM-004`	Principal wildcard sans Condition
Console sans MFA	`IAM-005`	Login profile présent, liste MFA vide
Bucket en écriture publique	`S3-001`	Public write effectif
Bucket en lecture publique	`S3-002`	Public read effectif
BPA incomplet	`S3-003`	au moins un contrôle effectif désactivé

Cas du laboratoire	Détection attendue	Validation logique
Volume détaché	`ORPHAN-001`	available + aucun attachment
EIP libre	`ORPHAN-002`	aucune association
ENI détachée	`ORPHAN-003`	available + aucun attachment
Bucket privé / ressources attachées	aucun finding correspondant	contrôle des faux positifs

6.4.2 Pour Azure :

Table 6.8 – Matrice de validation des détections Azure

Cas du laboratoire	Détection attendue	Validation logique
Owner au niveau subscription	`AZ-RBAC-001`	rôle Owner + scope subscription
Contributor au niveau subscription	`AZ-RBAC-002`	rôle Contributor + scope subscription
Storage Account autorisant le public	`AZ-STG-001`	`allowBlobPublicAccess=true`
Container public	`AZ-STG-002`	access level `blob` ou `container`
RDP/SSH publics	`AZ-NET-001`	Inbound Allow + source publique + port administrateur
NIC publique	`AZ-NET-002`	référence vers une Public IP découverte
Disque non attaché	`AZ-ORPH-001`	non référencé et sans `managedBy`
NIC non attachée	`AZ-ORPH-002`	non référencée par une VM
Public IP non attachée	`AZ-ORPH-003`	non utilisée par une NIC attachée
Reader RG / stockage privé / ressources utilisées	aucun finding équivalent	contrôle des faux positifs

Cette matrice montre également pourquoi Discovery et Analysis sont séparés. Discovery ne connaît pas le résultat attendu : il enregistre uniquement les propriétés. Analysis applique ensuite les règles aux données. La même donnée découverte peut être utilisée par plusieurs règles, par exemple la Public IP Azure sert à détecter une NIC publiquement exposée mais aussi à déterminer si une adresse est orpheline.

La présence de contrôles négatifs est essentielle. `crl-private-data`, `crl-safe-user`,

le volume AWS attaché, `stprivatebackups`, les rôles Reader ou le disque Azure utilisé doivent apparaître dans l'inventaire sans produire les findings correspondants. Cette propriété confirme que l'outil n'est pas un simple générateur de résultats basés sur le nom des ressources du laboratoire.

La dernière validation concerne le reporting. Après un scan complet, la chaîne est considérée fonctionnelle lorsque les quatre fichiers principaux existent et que `report.html` reflète les mêmes findings que les fichiers JSON. Le rapport n'est donc pas une source de vérité indépendante : il est la représentation finale du résultat structuré produit par le pipeline.

6.5 Constats principaux

Les scénarios de validation mettent en évidence plusieurs constats. Sur AWS, les risques les plus importants proviennent des autorisations IAM très larges et des buckets S3 exposés en lecture ou en écriture. Les ressources orphelines présentent généralement une criticité plus faible, mais leur détection reste utile pour identifier des actifs inutilisés et améliorer la visibilité sur l'environnement.

Sur Azure, les affectations Owner ou Contributor au niveau de la subscription sont davantage prioritaires que des rôles similaires limités à un Resource Group, ce qui confirme l'importance du scope dans l'analyse RBAC. L'exposition publique du stockage et des ports d'administration constitue également une surface de risque significative. Les relations entre VM, NIC, Public IP et disques montrent enfin que la détection d'assets orphelins dépend d'une découverte correcte des liens entre ressources, et non uniquement de leur présence dans l'inventaire.

Dans les deux implémentations, les contrôles négatifs confirment l'intérêt de séparer la découverte de l'analyse : une ressource peut être inventoriée sans être considérée comme vulnérable. Le projet met ainsi en évidence qu'une évaluation exploitable doit associer données techniques, contexte, règles explicites et mécanisme de priorisation.

Chapitre 7

Conclusion générale

Le projet **Cloud Risk Assessment Tool** aboutit à deux implémentations fonctionnellement cohérentes d'un même modèle d'évaluation Cloud. La première cible AWS à travers LocalStack et `boto3`, la seconde cible Azure à travers Topaz, une couche REST locale et `topaz-sdk`. Malgré les différences entre les providers, les deux versions suivent la même chaîne : **Discovery -> Analysis -> Scoring -> Reporting**.

Le principal apport du projet réside dans cette séparation des responsabilités. Discovery collecte les faits techniques sans décider de leur dangerosité. Analysis applique des règles explicites et traçables. Scoring ajoute la notion de priorité. Reporting transforme enfin le résultat en fichiers utilisables par un humain ou par une automatisation.

Sur AWS, l'outil couvre six contrôles IAM, trois contrôles S3 et trois règles d'assets orphelins. Le laboratoire de référence active volontairement dix findings et conserve plusieurs ressources sûres pour contrôler les faux positifs. Le modèle de scoring sur 10 combine impact, likelihood, exposition et tags de contexte.

Sur Azure, l'outil couvre trois règles RBAC, deux règles Storage, trois règles d'assets orphelins et deux contrôles réseau complémentaires. Le laboratoire construit des identités, Role Assignments, comptes de stockage, containers, ressources réseau et compute afin de produire un scénario cohérent de bout en bout. Le scoring sur 100 combine une sévérité initiale avec des facteurs d'exposition, privilège, sensibilité et exploitabilité.

L'utilisation d'environnements fictifs apporte un cadre reproductible pour le développement et la démonstration. Elle permet de réinitialiser rapidement les ressources, de connaître les risques introduits et d'itérer sur les règles sans dépendre d'un compte Cloud externe. Cette approche reste toutefois un environnement de validation : avant une utilisation sur un système de production, les permissions, la couverture API, la gestion des erreurs et les contrôles de sécurité devraient être validés sur des environnements réels maîtrisés.

L'architecture actuelle constitue une base extensible. De nouvelles règles peuvent être ajoutées dans les modules Analysis, de nouvelles ressources peuvent enrichir Discovery et d'autres modèles de scoring ou formats de sortie peuvent être introduits sans modifier

l'ensemble du pipeline. Le projet remplit ainsi son objectif initial : fournir un outil technique capable de découvrir un environnement Cloud fictif, détecter des configurations à risque, les prioriser et produire un résultat exploitable.

Perspectives

L'architecture développée constitue une base pouvant être enrichie sans remettre en cause le pipeline existant. Une première perspective consiste à étendre la couverture de Discovery et d'Analysis à d'autres services Cloud et à de nouvelles familles de contrôles, par exemple des configurations réseau supplémentaires, la journalisation, le chiffrement ou d'autres mécanismes de gestion des secrets et des identités.

Une deuxième évolution serait de rapprocher davantage l'outil des environnements Cloud réels. Cela nécessiterait notamment une gestion plus complète des mécanismes d'authentification, des permissions minimales de lecture, de la pagination, des quotas, des erreurs partielles et des variations régionales. Les laboratoires locaux resteraient utiles pour les tests automatisés, tandis que des environnements réels maîtrisés permettraient de valider la compatibilité opérationnelle.

Le modèle de scoring pourrait également être harmonisé entre AWS et Azure afin de faciliter les comparaisons multi-Cloud. Une normalisation commune des facteurs de risque, éventuellement enrichie par la criticité métier, l'exposition Internet et la sensibilité des données, permettrait d'obtenir une priorisation plus homogène.

Enfin, le reporting pourrait évoluer vers une interface centralisée avec historisation des scans, comparaison de l'évolution de la posture de sécurité, filtres par provider et sévérité, ainsi qu'un mécanisme d'export vers d'autres outils de suivi ou de traitement des vulnérabilités.

Annexes

Les annexes du livrable sont fournies sous la forme des **dossiers complets des deux projets compressés en ZIP**, accompagnés de la **présentation du projet**. Les archives contiennent le code source correspondant ainsi que les fichiers README nécessaires à la compréhension et à l'exécution des projets.

— **Projet AWS** : archive ZIP contenant `AWS-risk-tool` et sa documentation README / guide d'exécution.
— **Projet Azure** : archive ZIP contenant `azure-risk-tool` et sa documentation README / guide d'exécution.
— **Présentation du projet** : support de présentation synthétisant le contexte, l'architecture, la démarche adoptée et les principaux résultats du projet.

Le code source complet n'est pas reproduit dans le corps du rapport. Seuls les extraits utiles à la compréhension de l'architecture ou de la logique de détection y sont présentés.

Webographie / Bibliographie

[1] DATAPROTECT, *Site officiel*, https://www.dataprotect.ma/

[2] Python Software Foundation, *Python 3 Documentation*, https://docs.python.org/3/

[3] Docker, *Docker Documentation*, https://docs.docker.com/

[4] Amazon Web Services, *Boto3 – AWS SDK for Python Documentation*, https://docs.aws.amazon.com/boto3/latest/

[5] LocalStack, *LocalStack Documentation*, https://docs.localstack.cloud/

[6] Amazon Web Services, *AWS Identity and Access Management Documentation*, https://docs.aws.amazon.com/iam/

[7] The Cloud Theory, *Topaz – Local Azure Emulator Documentation*, https://topaz.thecloudtheory.com/

[8] Microsoft, *Microsoft Azure Documentation*, https://learn.microsoft.com/azure/